C Programming — Functions

Common Errors, Causes & Solutions | Lecture Notes

Overview

In our previous lecture, we covered an introduction to functions — what functions are, why they are used, how they work, and how they are declared and implemented. Now we will take the next step and learn how to write and use functions correctly. We will understand the mistakes that programmers commonly make, why those errors occur, and how to fix them.

Error 1 — Implicit Declaration of Function

This is one of the most common errors beginners encounter when calling a function before it is declared. The C compiler reads code line by line from top to bottom. If you call a function inside `main()` before the compiler has seen any declaration for it, it does not know the function's return type or parameter list — so it raises an **implicit declaration** error.

Implicit means: *not directly stated or declared.* The compiler is forced to make an assumption about the function, which is dangerous and non-standard in C99+.

Buggy Code

```
#include <stdio.h>


int main()

{

    int area_rect = area_ofrectangle(15, 3);

    printf("%d", area_rect);

    return 0;

}
```

```
int area_ofrectangle(int l, int b)

{

    return l * b;

}
```

Compiler Output

```
ERROR!


/tmp/41btNy8UID/main.c: In function 'main':

/tmp/41btNy8UID/main.c:4:21: error:

    implicit declaration of function 'area_ofrectangle'

    [-Wimplicit-function-declaration]


4 |     int area_rect = area_ofrectangle(15, 3);

  |                     ^~~~~~~~~~~~~~~~


=== Code Exited With Errors ===
```

Why does this happen?

- The C compiler reads source code strictly from top to bottom.
- When it reaches the call `area_ofrectangle(15, 3)` inside `main()`, it has not yet seen the function definition.
- With no information about the function's signature, the compiler cannot verify argument types or the return type — so it throws an implicit declaration error.

Solution — Add a Function Prototype

Declare a **function prototype** before `main()`, right after your `#include` directives. A prototype tells the compiler: *"This function exists, here is its return type and parameter list."* The actual definition can still appear after `main()`.

```
#include <stdio.h>
```

```
/* Function prototype — declared BEFORE main() */

int area_ofrectangle(int l, int b);


int main()

{

    int area_rect = area_ofrectangle(15, 3);

    printf("%d", area_rect);

    return 0;

}


/* Actual function definition — placed AFTER main() */

int area_ofrectangle(int l, int b)

{

    return l * b;

}
```

Note: Alternatively, move the entire function definition above main() and you won't need a separate prototype at all — the definition itself acts as one.

Error 2 — Mismatched Return Type

If the value a function returns does not match its declared return type, the compiler may warn or silently truncate/cast the value, producing wrong output.

Buggy Code

```
#include <stdio.h>


int divide(int a, int b);   /* declared to return int */


int main()
```

```
{
    printf("%f", divide(7, 2));   /* expects float output */
    return 0;
}

int divide(int a, int b)
{
    return (float)a / b;   /* float result truncated to int */
}
```

Why does this happen?

- The function is declared as `int`, so the compiler truncates the `float` result to an integer before returning.
- Using `%f` with an `int` return causes undefined behaviour — output will be garbage or 0.

Solution — Match the return type

```
#include <stdio.h>

float divide(float a, float b);   /* correct return type */

int main()
{
    printf("%.2f", divide(7, 2));   /* output: 3.50 */
    return 0;
}

float divide(float a, float b)
{
```

```
    return a / b;

}
```

Error 3 — Wrong Number of Arguments

Passing fewer or more arguments than a function expects causes a compilation error or unexpected runtime behaviour.

Buggy Code

```
#include <stdio.h>


int add(int a, int b, int c);


int main()

{

    printf("%d", add(5, 10));   /* only 2 args, needs 3 */

    return 0;

}


int add(int a, int b, int c)

{

    return a + b + c;

}
```

```
error: too few arguments to function 'add'
```

Solution — Pass the correct number of arguments

```
printf("%d", add(5, 10, 15));   /* output: 30 */
```

Error 4 — Missing return Statement in Non-void Function

If a function is declared to return a value but has no `return` statement, the compiler will warn and the function will return garbage — a random value left on the stack.

Buggy Code

```
#include <stdio.h>


int square(int n);


int main()

{

    printf("%d", square(4));

    return 0;

}


int square(int n)

{

    int result = n * n;

    /* forgot: return result; */

}
```

```
warning: control reaches end of non-void function [-Wreturn-type]

Output: (garbage value)
```

Solution — Always return the correct value

```
int square(int n)

{

    int result = n * n;
```

```
    return result;   /* output: 16 */

}
```

Error 5 — Expecting a Function to Modify a Variable (Pass by Value)

C passes arguments **by value** — the function receives a *copy*. Modifying the parameter inside the function does NOT affect the original variable in the caller.

Buggy Code

```
#include <stdio.h>


void double_it(int x);


int main()

{

    int num = 5;

    double_it(num);

    printf("%d", num);   /* programmer expects 10, gets 5 */

    return 0;

}


void double_it(int x)

{

    x = x * 2;   /* only modifies the local copy */

}
```

Solution — Use a pointer to modify the original variable

```
#include <stdio.h>
```

```c
void double_it(int *x);


int main()

{

    int num = 5;

    double_it(#);

    printf("%d", num);   /* output: 10 */

    return 0;

}


void double_it(int *x)

{

    *x = (*x) * 2;   /* modifies the original via pointer */

}
```

Quick Reference — Error Summary

#	Error	Cause	Fix
1	Implicit declaration	Function called before declaration	Add prototype before main()
2	Mismatched return type	Return value type != declared type	Match return type in prototype & definition
3	Wrong argument count	Too few or too many args passed	Check prototype and pass correct count
4	Missing return statement	Non-void function with no return	Add return with the correct value
5	Pass by value confusion	Trying to modify caller's variable	Use pointer parameter instead